SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.* (BL4)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 30**

Question Count: 5

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the student: _____

Assessment Tasks Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.
3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing

structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.
5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Reverse Engineering Task Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of underlying big data architecture	Good understanding	Partial understanding	Poor understanding
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

1. Big Data Platform for Website Clicks, Mobile Interactions and Transactions

Introduction

A platform handling billions of website clicks, mobile interactions, and transactions needs a **distributed big data architecture** because a single system cannot handle such a huge workload.

1. Distributed Storage

The platform may use **HDFS or cloud object storage** such as S3. Data is divided into blocks and stored across multiple servers. Replication provides fault tolerance.

2. Data Ingestion

Tools such as **Apache Kafka, Flume, or NiFi** can collect data from websites, mobile applications, and transaction systems.

Website/Mobile → Kafka → Data Storage

3. Batch Processing

Apache Spark or Hadoop MapReduce can process large historical datasets for sales analysis, customer behavior, and reporting.

4. Data Types

Structured data such as customer IDs, orders, prices, and payments can be stored in relational databases or warehouses.

Semi-structured data such as JSON, clickstream logs, and application logs can be stored in a data lake or NoSQL database.

5. Stream Processing

Kafka, Spark Streaming, or Apache Flink can process events in real time. For example, a customer's product click can immediately update recommendations.

6. Scalability and Partitioning

The system uses **horizontal scaling**, where more servers are added. Data can be partitioned by:

- Date
- Customer
- Region
- Product

Partitioning improves processing speed.

2. E-Commerce Platform for Customer Behaviour and Recommendations

Introduction

An e-commerce platform generates continuous events such as product views, cart additions, abandoned checkouts, and payments. These events can be processed to provide real-time personalization.

1. Event Streaming

Apache Kafka can collect events such as product views, cart additions, payments, and checkout abandonment.

2. Customer Profiling

Customer information can be stored in **Cassandra, MongoDB, Redis, or relational databases**. Profiles may contain purchase history, browsing history, preferences, and location.

3. Recommendation System

Machine learning models analyze customer activity to recommend suitable products.

Customer Activity → Feature Processing → ML Model → Product Recommendation

4. Data Synchronization
Customer ID, session ID, product ID, and order ID can connect browsing, session, and transaction information.
Kafka and stream-processing tools help synchronize these events in real time.
5. Caching
Redis can store frequently accessed information such as:
- Shopping carts
- Product details
- Customer sessions
- Recommendations

This reduces database load and improves response time.
6. Data Integration
Structured order data can be stored in relational databases or warehouses. Semi-structured browsing data such as JSON logs can be stored in a data lake. They can be merged using common IDs.
7. Scalability and Fault Tolerance
The system can use:
- Kafka partitions
- Database replication
- Sharding
- Load balancing
- Multiple processing nodes

3. Big Data Healthcare System
Introduction
Healthcare systems process patient histories, medical images, wearable sensor data, and prescriptions. Since these are different types of data, a **hybrid database architecture** is required.
1. Hybrid Storage
Structured data such as patient IDs, prescriptions, diagnoses, and laboratory results can be stored in relational databases.
Medical images and documents can be stored in distributed or cloud object storage. NoSQL databases can store semi-structured sensor data.
2. Data Integration
Hospitals, laboratories, insurance companies, and pharmacies can exchange data using integration tools and healthcare standards such as **HL7 and FHIR**.
DICOM can be used for medical images.
3. Streaming
Wearable devices continuously generate heart rate, blood pressure, and other readings. **Kafka, Spark Streaming, or Flink** can process these streams.
4. Machine Learning
ML can support:
- Disease prediction
- Patient risk analysis
- Medical image analysis

- Readmission prediction
- Abnormality detection

5. Security

Healthcare information requires strong protection through:

- Encryption
- Authentication
- Role-based access control
- Data masking
- Audit logs

6. Compliance

Only authorized users should access patient information. Data access and modifications should be recorded for auditing and compliance.

7. Analytics Pipeline

A possible pipeline is:

Hospital/Wearable/Lab → Data Integration → Data Lake → Spark → ML → Clinical Dashboard

8. Clinical Decision Support

Analytics can provide doctors with risk scores, abnormal test alerts, and patient monitoring information to support clinical decisions.

4. Smart City Big Data Platform

Introduction

A smart city continuously collects data from traffic sensors, CCTV systems, weather stations, buses, and trains. A distributed architecture is needed to process this data efficiently.

1. Edge Computing

Sensors can send information to nearby edge devices.

Sensor → Edge Gateway → Central Platform

Edge processing reduces network traffic and provides faster responses.

2. Distributed Messaging

Apache Kafka can collect traffic, weather, CCTV, and public transport events. Kafka partitions allow many events to be processed simultaneously.

3. Central Storage

A data lake, HDFS, or cloud storage can store large amounts of historical sensor data.

4. Geospatial Analytics

Location-based analytics can identify:

- Traffic congestion
- Accident locations
- Vehicle speed
- Road usage
- Public transport routes

PostGIS or similar geospatial databases can support location queries.

5. Time-Series Databases

Databases such as **InfluxDB or TimescaleDB** can store time-based sensor information such as traffic counts, temperature, and speed.

6. Real-Time and Batch Processing
Spark Streaming or Flink can process current traffic data and predict congestion.
Batch processing can analyze historical data for long-term city planning.
7. Security
Security measures include:
- Encryption
- Authentication
- Role-based access control
- API security
- Audit logging

5. Healthcare Analytics System
Introduction
A healthcare analytics system combines patient records, wearable readings, laboratory reports, and appointment information. It requires secure storage and distributed processing.
1. Hybrid Storage
Structured information such as patient details, appointments, prescriptions, and laboratory results can be stored in relational databases.
Reports, sensor data, and medical documents can be stored using NoSQL databases, data lakes, or object storage.
2. ETL Pipeline
The ETL process is:
Extract → Transform → Load
Data is collected from hospitals and clinics, cleaned, standardized, and loaded into the analytics platform.
3. Interoperability
Healthcare systems can use **HL7 and FHIR** for exchanging clinical information. **DICOM** can be used for medical images.
4. Real-Time Monitoring
Wearable devices can continuously send patient readings.
Wearable → Kafka → Stream Processing → Alert
Abnormal readings can generate immediate alerts.
5. Predictive Analytics
Machine learning can predict:
- Disease risk
- Patient deterioration
- Hospital readmission
- Treatment outcomes
6. Distributed Processing
Apache Spark can process millions of patient records across multiple servers. This helps perform population-level studies and identify health trends.